

Reading While Writing Without Waiting

2018-03-06 07:06:18

Original: <https://nicknash.me/2018/03/06/reading-while-writing-without-waiting/>

1 Reading and Writing

It's fairly common to have a data structure that is occasionally mutated but often read, with the writing and reading occurring on different threads. This type of scenario can arise during initialization or in a loading phase. A bunch of data structures are built, with potentially many mutating threads, and from then on (or for the next long while) are only read, again potentially from many threads. The usual ways of dealing with this are reader-writer locks, or occasionally a lock-free approach, or perhaps even RCU.

A while ago I wondered if there were other *generic*, efficient ways to deal with this scenario, and I happened upon the very interesting so-called Left-Right construction. I thought it was so nice I'd even resurrect my blog to describe it!

2 Left-Right to the Rescue

Left-Right is a genuinely new way to deal with concurrent reading and writing of data structures, first described by Pedro Ramalhete and Andreia Craveiro (see here (<https://github.com/pramalhe/ConcurrencyFreak>) for a paper). After I first read about it, I decided I'd throw together a little C# implementation, as one apparently didn't exist. I must also say that Pedro Ramalhete was extremely helpful and friendly in answering my queries about Left-Right.

But before describing how the Left-Right construction works, I'll motivate things a little by revisiting a simpler generic solution to the problem of concurrent reading and writing of a data structure, namely lock-free copy-on-write. There are other examples that could also be used to motivate things, like an optimised reader-writer lock. Copy-on-write is handy to describe because it's fairly self-contained and has similar-ish progress guarantees to Left-Right, and perhaps most importantly it is similarly an extremely general construction.

3 Copy-on-Write

Making a data structure thread-safe via copy-on-write simply copies the entire data structure when a write occurs. This sounds expensive, but in some situations where the read-path latency is critical, it can be justified.

For a single writer, and many readers, an example of copy-on-write is something like this:

```

List data;

void Write(int v)
{
    var original = Volatile.Read(ref _data);
    var copy = new List(original);
    copy.Add(v);
    Volatile.Write(ref _data, copy);
}

int Read(int idx) => Volatile.Read(ref _data)[idx];

```

This is neat in a few ways:

- It's very simple.
- It's generic, and works with any data structure.
- Reads and writes can happen concurrently.
- It relies only on a store-release by the writer and a load-acquire by the reader. So, on say x86/x64, the thread synchronisation is free.
- If we are mostly reading (or after an initialisation phase, always reading) the reading path is very efficient, hardly different than when only a single thread is involved.

It also has some disadvantages:

- For large data structures, writing is very inefficient. In this case, an (amortized) $\Theta(1)$ add to end of list changes to $\Theta(n)$ time.
- It creates garbage. Or maybe worse it *requires* a garbage collector of some kind. Something has to be responsible for freeing the memory associated with data when it is replaced with the copy, and this must only happen when all readers are finished with the pre-write version.
- In the presence of multiple writers, scaling is very poor - worse than blocking - as described next.

In the presence of multiple writers, things gets worse. To support multiple writers, we need to do something like this:

```

void Write(int v)
{
    while(true)
    {
        var original = x;
        var copy = new List(original);
        copy.Add(v);
        if (Interlocked.CompareExchange(ref _data, copy, local) == local)
        {
            break;
        }
    }
}

```

For workloads that are not overwhelmingly writers or larger lists, this scales very poorly indeed. In fact, simple blocking completes more quickly as we'll see next.

4 A Micro-benchmark of Copy-on-Write

To get a slightly better feel for how copy-on-write behaves, a micro-benchmark can tell us a little. The graphs below compare three simple list implementations:

- **MultipleWriterCOWList**: This is the copy-on-write list described above, and is the one we're really talking about.
- **LockList**: This is a list that is protected by a simple lock, i.e., it's fully blocking for reads and writes.
- **RWLockList**: This is a list protected by a reader-writer lock (using the `ReaderWriterLockSlim` in C#).

The following two graphs show some of the higher percentiles for the latency of an operation (either a read or write) on the lists, for 4 threads, a 100-element list and a workload of 95% reads (left), and 5% reads (right):

[gallery ids="589,588" type="rectangular"]

Looking at the left plot, the copy-on-write approach fairs quite well. The right plot reminds us how much copy-on-write depends on the number of writes being comparatively small though. Incidentally, the fact that the reader-writer lock is so bad is interesting too, but a topic for another day!

Next we can look at how copy-on-write scales with the number of threads involved, jumping up to 20 threads, things don't go well for it:

[gallery ids="586,585" type="rectangular"]

The left plot shows something any lock-free algorithm is vulnerable to: outliers. Lock-freedom guarantees *some* thread makes progress at each step, but makes no guarantees about fairness or starvation. In the case of the left plot, above about the 95th percentile, the blocking list ("LockList") has lower latency. On the right, the setup is the same except it consists of only 5% reads. In this case, the y-axis is the logarithm of latency. In other words the copy-on-write list is a scaling *disaster*.

The last obvious knob we can turn is the capacity of the list in question. The copy-on-write list must do more work the larger lists. Switching to 1000-element lists with 95% reads for 4 threads (left) and 20 threads (right) shows the copy-on-write list again is vulnerable to outliers, more so than a trivial fully blocking list ("LockList"):

[gallery ids="587,584" type="rectangular"]

Aside from the latency distributions, we can look at the throughput of the various types of list:

[gallery ids="583,600" type="rectangular"]

[gallery ids="581,580" type="rectangular"]

The four graphs above show that for a 1000-element list, the copy-on-write list is always lower throughput than simpler blocking "LockList" alternative (again, why the reader-writer lock is so poor is a topic for another day!). Moreover, its throughput is only higher for a small, 100-element list for workloads that are overwhelmingly writes.

All the same, copy-on-write can still be attractive in the case where there is just a *single* writer. In that case, there is no question of outliers due to multiple writes competing (although they might still occur if the data structure was large).

It seems natural to wonder if there is some intermediate solution between reads being as cheap as in the single-threaded case, and writing involving copying the entire data structure. That is, is there a solution where reads involve more work than copy-on-write, but writes require less. It turns out that the Left-Right construction is an example that fills this gap.

5 A First Look at Left-Right

Copy-on-write creates copies of the data structure only at the moment when writes occur, instead of this, what if we always maintained two copies of the data structure?

A write operation would always write to both instances, but it would only write to an instance when it can be certain that no reader is reading that instance.

This is basically the first idea of Left-Right. The remaining details are the ingenious mechanism for coordinating reading and writing threads cheaply and with strong progress guarantees.

Left-Right combines the following surprisingly powerful guarantees:

- It's generic, works with any data structure, and doesn't affect the time complexity of any operations.
- Reads and writes can happen concurrently.
- Reading is always wait free.
- No allocation is required, so no garbage is generated, and no garbage collector is required.
- A writer cannot be starved by readers.

Some disadvantages of Left-Right are:

- It's more complicated than copy-on-write.
- It requires performing every mutation of the data structure twice.
- Multiple writers block each other (but actually, this scales better than copy-on-write, and at least matches a simple blocking solution).

6 Next Time

This post is getting a bit long, but hopefully it has piqued your interest in Left-Right!

There is an evolving landscape of solutions to the type of problem addressed by Left-Right (as well as simple old copy-on-write). If you're interested in an overview and a 'state of the art' approach, Chen et al's 2016 paper Fast Consensus Using Bounded Staleness for Scalable Read-Mostly Synchronization (<http://ipads.se.sjtu.edu.cn/lib/exe/fetch.php?media=publications:consensus-tpds16.pdf>) is a great read.

In some future posts I hope to describe a C# implementation of Left-Right, and how to validate the implementation using RelaSharp (<https://github.com/nicknash/RelaSharp>). Be sure to

check out the blog (<http://concurrencyfreaks.blogspot.ie>) of Perdro Ramalhete and Andreia Craveiro where you can find lots of information on the creation of Left-Right.

On the off chance you'd like to see the (not very nice!) bench-marking code used for copy-on-write it's available at <https://github.com/nicknash/COWBench> (<https://github.com/nicknash/COWBench>)